PFP III: Admitting Failure Detectors and Testing Their Effects

Jeremy H. Carroll

2026-10-02 · Version 0.1.0 · Methods paper · Author-review revision

Code baseline: 52cb5457f6c485ba548cfd292f6012a941ed22cf

Abstract

The Proposal Fidelity Protocol (PFP), previously called the Symbol Game, treats generated text as a proposal whose admissibility depends on a declared checker. A failure detector adds another decision: whether a recurring pattern should change how subsequent proposals are handled. That decision can prevent a known mistake, but it can also suppress a valid unfamiliar answer. This paper specifies detector admission, separates diagnostic findings from route verification, and presents a proposed study protocol for effectiveness and false suppression: blocking a proposal that an independent checker judges valid. Four unadmitted phrase detectors serve as worked examples; one of them, as first written, also matched a true statement about the attractive Hubbard model. The implementation contribution is a source-audited description of a bounded host and a two-signal filter, with explicit transaction rollback, fuel consumption, and receipt replay. At the audited revision, the canonical Demo 3 host returns `NO_NATIVE_PORT` for all three witness routes. Its successful protocol checks therefore cannot establish witness acceptance or a live treatment effect. A worked batch shows how this refusal preserves prior semantic state while recording the failed attempt. The proposed experiment compares a frozen detector policy with matched reflection feedback and requires independent route outcomes, randomized allocation, and informative safety intervals. Appendix A, new in version 1.2.0, establishes three small exact lemmas: a second signal computed from the first reduces the two-signal table to a one-signal detector, a Hill-type "quorum" is a threshold on a sum and bounds no number of agents, and screening a detector against a self panel is sound for every panel exactly when the executed predicate implies the screened predicate. It audits the repository against them and describes one component that realizes the admission record of Section 3. No new provider trial or general reduction in hallucination is reported.

1. Question, contribution, and source boundary

Consider a proposer asked to repair a small symbolic object. A checker can determine whether one proposed repair satisfies a finite contract. A detector instead recognizes a possible reason that an attempt may fail: a supplied verification flag, incomplete coverage, a reused invalid witness, or a mutation outside the allowed carrier. The research question is whether admitted detectors improve the rate of independently verified solutions without unacceptably suppressing valid proposals.

The contribution has three parts. First, we give a detector admission procedure whose unit is a reproduced defect together with a valid near miss. Second, we identify what the current host actually enforces and provide a complete refusal example. Third, we specify an experiment that could distinguish a detector policy from ordinary reflection. Admission procedure and experiment design are proposals; the host branches and test assertions are inspected implementations.

The implementation audit is pinned to repository commit 52cb5457f6c485ba548cfd292f6012a941ed22cf. Repository paths below refer to that revision. Paths beginning with `code/` refer to a small re-implementation included in this deposit (Section 4.1), which is not the canonical host. Fresh

execution results belong to a separately dated receipt and must identify their source bytes. This manuscript does not reuse historical passing counts as current execution evidence.

PFP is used here as a verifier-relative protocol. A checked claim always has a carrier, allowed operations, a checker, and a scope. A detector finding does not establish that a proposition is false in every interpretation. A model identity different from the proposer is also insufficient for independence: the outcome checker must evaluate the declared property rather than repeat a generated judgment.

2. Related work

Reflexion retains linguistic feedback in episodic memory to inform later agent attempts. Its feedback can originate from external or internally simulated sources. This makes a reflection control necessary when studying a more structured detector intervention; improvement over plain prompting would not identify which feedback mechanism helped. Shinn et al., 2023

Self-Refine iterates generation, feedback, and revision using the same language model. It supplies a useful comparison for a proposal-improvement loop. The present design adds separately evaluated witness outcomes and records when a filter would reject a proposal that the route checker accepts. Madaan et al., 2023

These papers establish prior methods, not the performance of PFP. The present study has not measured an advantage over them. Biological words in implementation names identify software components only. The "danger gate" is the two-signal decision table of Section 3. The "thymus" module rejects a candidate detector whose pattern matches any entry of the self panel, a fixed list of strings treated as valid. No claim about biological fidelity or general immunity follows from those names.

3. Detector objects and admission

A proposed detector record should contain its identifier, version, applicable route, input schema, executable predicate, defect code, discovery provenance, and action policy. It must bind a positive witness that reproduces the named defect and at least one independently valid near miss. A prose warning without these artifacts remains a suggestion.

Admission proceeds through five checks. The host first reparses the offending proposal and reproduces the defect. It then minimizes the example sufficiently to identify which condition failed. Next it constructs valid cases close to the failure boundary, including alternative notation and relabeling where allowed. The detector is evaluated on both classes and its intended action is frozen. Finally, the detector and fixture digests are bound into the experiment manifest. A modified predicate or broader route scope requires a new admission record.

This procedure is intentionally stricter than counting repeated strings in a failure log. Repetition may reflect a duplicate task, a shared prompt, or a checker bug. One thousand stored rows are not one thousand independent examples. The repository's self panel at the baseline has 1,000 rows but only 40 distinct base contents, ten in each of four categories. That is a finite regression corpus, not a statistical estimate of false suppression in open use. See the regression suite.

The actual two-signal gate offers a small operational model. Signal 1 records a pattern match. Signal 2 records a supplied structural defect, severity, and an evidence identifier. With neither signal the decision is `TOLERATED`. Signal 1 alone yields `DIAGNOSTIC_ONLY`. Both signals produce a refusal; a high

or critical defect alone also produces a refusal. A low or medium defect alone remains diagnostic. The implementation computes this decision table; it does not independently prove the supplied defect or authenticate its evidence identifier. The table is only as informative as the independence of its two inputs: Appendix A shows that when Signal 2 is computed from Signal 1, it reduces to a one-signal detector.

For example, a pattern matching the word `CUT` may occur in a harmless explanation. Without structural damage, refusing it would confuse mention with execution. Pair that match with a host-established protected-field mutation and the declared policy refuses. The policy benefit is plausible, but whether these two inputs are accurate in a deployed pipeline remains a separate obligation. The conjunction tests exercise the decision table on constructed signals. They do not measure natural-language classification accuracy.

Detector actions should be distinguished: a finding records an observation; a hold requests further evidence; a refusal rejects one proposed transition. None grants permission to repair the object automatically. The repair proposal and the defect report remain separately inspectable.

3.1 Four candidate phrase detectors

The package contains four candidate detectors, written as phrase lists in `code/canaries.py` with a Rust port in code/rust/src/canaries.rs. A detector reports a finding when one of its phrases occurs in the lower-cased input. Such a finding is Signal 1 in the gate above. A phrase match cannot check a physical or mathematical condition, and it cannot tell an assertion from a quotation, a question, or a negation: the sentence "There is no particle-hole symmetry on triangle plaquettes" matches the AM-FSP-2 phrase "particle-hole symmetry on triangle". None of the four detectors has passed the admission procedure above. Each lacks a defect reproduced by an independent checker and a set of valid near misses evaluated by that checker. They are listed as examples of what admission has to examine.

The identifiers come from an earlier harvest of responses to six physics prompts. The harvest file, research/mssn_sn_tower/demos/antimorphism_farm/harvest/antimorphism_harvest.jsonl, has 112 rows. In 102 of them the response is text written in advance by the authors, copied from the scripted fields of the prompt specifications in omega_engine_epistemic_immune_system/infrastructure/mssn_domains.py. The other 10 rows are responses from one model, qwen/qwen-2.5-7b-instruct. A separate, prompt-specific rule set in omega_engine_epistemic_immune_system/infrastructure/mssn_classifier.py labelled the rows with these identifiers, and no independent checker reviewed the labels. The harvest therefore does not measure how often models make these errors. It is also not evidence about the package detectors, which reuse the identifiers but not the rules.

Each entry below gives the targeted claim, a fixture sentence used by the package tests, and what the phrase list can and cannot do.

1. **AM-FSP-8 (unrestricted solver claim).** Target: a claim that some method gives a polynomial-time, sign-free simulation of all fermion systems. Troyer and Wiese showed that the sign problem is NP-hard for a class of frustrated models, so a generic polynomial-time solution of it would solve every problem in NP in polynomial time Troyer and Wiese, 2005. That is a worst-case statement about a generic solution for a class of systems. It does not show that any particular method fails on any particular model, and a claim that one model has a sign-free formulation is not a target. Constructed fixture text (written by the authors, not observed model output): "Modern neural network quantum states and tensor networks have

overcome this limitation, providing an unrestricted polynomial-time sign-free solution for all fermion systems without requiring P = NP." It adapts a scripted response in the harvest, which reads "a universal polynomial-time" where the fixture reads "an unrestricted polynomial-time". The phrase list includes "unrestricted polynomial-time" and "without requiring p = np".

2. **AM-FSP-2 (particle-hole symmetry on non-bipartite lattices).** Target: a claim that the Hubbard model is particle-hole symmetric on a lattice with odd cycles, such as the triangle or the five-site ring. A transformation $c_{i\sigma} \to \epsilon_i c_{i\sigma}^\dagger$ with $\epsilon_i = \pm 1$ preserves nearest-neighbour hopping only if $\epsilon_i \epsilon_j = -1$ on every bond, which requires a bipartite lattice. On the triangle the model has no particle-hole symmetry at all: its hopping spectrum is $\{-2t, t, t\}$, which is not symmetric about zero. What requires a bipartite lattice is this particle-hole transformation, both where it relates the attractive and repulsive models and where it makes the model particle-hole symmetric at half filling. Sign-free simulation of the attractive model does not. With equal spin populations and real hopping, the attractive-U Hubbard model is sign-problem-free in determinant quantum Monte Carlo on any lattice, bipartite or not: the charge-channel decoupling makes the spin-up and spin-down determinants equal, so each weight is a square. Constructed fixture text (written by the authors, not observed model output): "For attractive U < 0, particle-hole symmetry holds on any lattice including the triangle and pentagon." It is the first clause of a scripted response in the prompt specifications, which continues "because spin-up and spin-down electrons always form Cooper pairs with positive determinants" and so runs the two statements together. The previous release of the package keyed this detector partly on the phrases "attractive u < 0" and "triangle and pentagon". The first also occurs in the true sentence "For attractive U < 0 with equal spin populations, determinant QMC is sign-problem-free on any lattice." A detector keyed on "attractive U … any lattice" would therefore flag a true statement, and any policy that refuses on such a finding would suppress it: a concrete instance of the false suppression this paper warns about. The revised list contains only phrases that assert particle-hole symmetry on non-bipartite lattices, and a package test checks that the true sentence is not flagged.

   The diagonal sign-gauge question is separate again. Non-bipartiteness alone does not obstruct a diagonal sign gauge. For example, the 3×3 Hamiltonian

$$H = \begin{pmatrix} 0 & 1 & -1 \\ 1 & 0 & 1 \\ -1 & 1 & 0 \end{pmatrix}$$

   has a triangle as its underlying graph, yet the diagonal sign gauge $S = \mathrm{diag}(1, -1, 1)$ makes every off-diagonal entry non-positive:

$$SHS = \begin{pmatrix} 0 & -1 & -1 \\ -1 & 0 & -1 \\ -1 & -1 & 0 \end{pmatrix}.$$

   Obstruction to a diagonal sign gauge requires a frustrated signed cycle, one whose gauge-invariant sign product $\prod_{(i,j)\in C} \mathrm{sgn}(-H_{ij})$ equals -1, rather than odd-cycle topology alone.

3. **AM-FSP-7 (Majorana positivity asserted without a condition).** Target: a claim that writing fermions in a Majorana basis, or using a continuous-time expansion, by itself secures positive Monte Carlo weights. Sufficient conditions for sign-problem-free Majorana quantum Monte Carlo are stated by Li, Jiang and Yao in terms of Majorana time-reversal symmetries Li et al., 2016 and by Wei, Wu, Li, Zhang and Xiang in terms of Majorana

positivity Wei et al., 2016. A phrase match cannot check whether a given Hamiltonian satisfies either condition; that would need a route-specific checker, which this package does not have. Constructed fixture text (written by the authors, not observed model output): "Continuous-time Majorana decomposition secures positive Monte Carlo weights." The phrase list includes "majorana decomposition secures positive" and four phrases about Majorana or Clifford lifts, mappings, and representations, such as "continuous majorana lift".

4. **AM-FSP-4 (meron cancellation asserted beyond its models).** Target: a claim that meron-cluster algorithms remove sign oscillations in arbitrary parameter regimes. The meron-cluster method cancels, in pairs, the contributions of configurations that contain a meron, a cluster whose flip changes the sign; it removes the sign problem for particular models whose cluster decompositions have the required properties Chandrasekharan and Wiese, 1999. A claim beyond those models is not supported by the method. Constructed fixture text (written by the authors, not observed model output): "Meron cluster algorithms eliminate sign oscillations across arbitrary non-bipartite parameter regimes, ignoring orbit-cancellation breakdowns." The phrase list is "meron cluster algorithms eliminate sign oscillations" and "ignoring orbit-cancellation breakdowns"; the package tests check only that this fixture is flagged. The previous release also filed group-averaging phrases such as "burnside lemma violation" under this identifier. Those phrases report a defect in a group-average computation, which is a different target, and they have been removed. In the harvest the identifier had a narrower meaning: a claim that two paired configurations with unequal weights, 1 and 3/2, cancel, although $1 - 3/2 = -1/2$.

The package tests check that each fixture sentence is flagged by its own detector only, that three clean controls (including the true attractive-U sentence) are not flagged, and that the negated sentence above is still flagged (code/tests/test_demo_3_host.py and code/tests/test_danger_gate.py). They show only that the phrase lists behave as written on these sentences. They measure no accuracy, sensitivity, or false-suppression rate.

4. The current host and its missing route seam

The canonical Demo 3 vocabulary declares six arm identifiers and five fixture identifiers across three routes:

Route	Intended witness examples	Current canonical verification
Restricted diagonal sign gauge	Vertex bits or an odd-parity cycle	`NO_NATIVE_PORT`
Exact rational cochain packet	Potential or separating left-null functional	`NO_NATIVE_PORT`
Finite signed path pairing	Partial sign-reversing path involution	`NO_NATIVE_PORT`

The use-case host supplies phases, fuel accounting, atomic transaction rollback, and hash-chained batch receipts. Each batch that the host processes costs one unit of fuel, charged before its moves are applied; the default initial fuel is 12. If the batch is refused, the semantic state (covered elements, witness candidate, phase) returns to its pre-batch value, but the fuel unit is not returned. A batch with a move after `VERIFY` or with a malformed element identifier is refused and charged in this way. An empty batch is also charged; it is recorded as `ACCEPTED` and changes no semantic state. Once fuel

is zero, the next batch is not applied: it is recorded as `FUEL_EXHAUSTED` and the host enters `TERMINAL`, after which batches are rejected with `REJECT_TERMINAL_STATE`. The number of processed batches is therefore bounded by the initial fuel.

The host's `state_digest` starts from a SHA-256 digest of the fixture and arm identifiers. Each batch extends it with SHA-256 over the prior digest, the proposed moves, and the host decision. The moves are serialized as JSON with sorted keys, no white space, and ASCII escapes; this is a declared convention, not a full RFC 8785 implementation. The digest does not cover fuel or the rest of the host state. The receipt verifier `verify_receipt_bytes` accepts only the canonical encoding of the expected 30-cell matrix (five fixtures $\times$ six arms). For each cell it recomputes the digest chain from the initial digest, checks the fuel sequence and the scripted decision sequence for the arm, and refuses a receipt that reports a terminal certificate or a witness type while the route port is missing. It therefore rejects `{}`, schema changes, chain discontinuities, and certificate claims. A SHA-256 chain without a key or an independently held anchor shows internal consistency; it does not authenticate who produced the receipt.

The host's `verify_route` function returns a typed missing-port refusal for any nonempty route name. The module does not call a native board or quotient action. While the seam is missing, returning `NO_NATIVE_PORT` is the appropriate outcome: it reports that verification is unavailable, not that a witness is wrong.

4.1 The package re-implementation

So that the refusal behavior can be run from this deposit alone, the package includes a small re-implementation: `code/host_engine.py` (host), `code/danger_gate.py` (two-signal table), `code/canaries.py` (phrase detectors), and `code/self_panel.py`, with a Rust port of the first three in `code/rust/`. It is not the canonical host, and its tests are illustrations, not integration evidence. It differs from the baseline in these respects:

- its default fuel is 11, and it enters `TERMINAL` as soon as a batch spends the last unit of fuel;
- its fixture list replaces `cochain_nonexact_cycle` with frustrated_square_diagonal, and it uses five route names, only one of which matches the baseline;
- it refuses an empty batch (`REJECT_EMPTY_BATCH`), and its `VERIFY` step requires only nonempty coverage and no witness;
- it represents the Arm D detector hold as a move type, not as a host method that records detector findings;
- its digests use a JSON initial record, the hexadecimal form of the move bytes, and UTF-8 output rather than ASCII escapes;
- its receipt verifier recomputes the digest chain and refuses certificate labels, but it does not check fuel or the scripted arm sequences, so an edited receipt whose digests are recomputed passes unless it carries a certificate label;
- its gate labels every two-signal refusal `QUARANTINED` and counts any supplied Signal 2 record as present, whereas the baseline gate counts a record only when its defect count is positive;
- `code/self_panel.py` builds a synthetic 1,000-row panel with the same shape (40 base strings, 25 tagged copies each) instead of loading the repository panel.

5. Worked example: a plausible witness that cannot be certified

Initialize `Demo3HostEngine("gauge_even_triangle", "T", initial_fuel=12)` in the baseline use-case host. The host begins in `INTAKE`, with empty coverage and no witness. Submit one batch containing:

```
[
  {"move_type": "COVER_ELEMENT", "payload": {"element_id": "elem_all"}},
  {"move_type": "PROPOSE_WITNESS",
   "payload": {"witness_type": "vertex_bits", "witness": [0, 1, 0]}},
  {"move_type": "VERIFY", "payload": {}}
]
```

The example displays the proposal fields; the tests construct typed `MoveProposal` objects. The host charges one unit of fuel, records coverage and the witness provisionally, then reaches `VERIFY`. Since the required route port is absent, it returns `NO_NATIVE_PORT` and rolls back the batch. Coverage is empty again, the witness is absent, the phase is `INTAKE`, and no terminal certificate exists. Remaining fuel is eleven: the charge is made before the moves are applied and is not returned on rollback. The step receipt records the phases before and after the batch, the proposed moves, the refusal decision, the prior and new digests, and the remaining fuel. The package re-implementation gives the same outcome for this batch; its test test_known_gauge_witness_still_refuses_and_rolls_back checks it, including the remaining fuel.

Now alter the batch by appending another move after `VERIFY`. The host refuses with `REJECT_MOVE_AFTER_VERIFY`, charges one unit, and rolls back the whole batch. A batch with a malformed element identifier is refused in the same way with `REJECT_MALFORMED_PAYLOAD`. An empty batch is charged one unit but changes no semantic state; the package re-implementation refuses it instead. After the twelfth charged batch, remaining fuel is zero; the next batch is not applied, is recorded as `FUEL_EXHAUSTED`, and leaves the host in `TERMINAL`.

For a novice, the distinction is practical: "the checker is unavailable" is an answer about the verification process. It is not an answer that the proposed mathematical witness is false. The retained attempt can later be replayed against the same protocol; testing the witness after a port is added requires a new, explicitly versioned evaluation.

6. Design for a prespecified effectiveness experiment

The experiment is conditional on implementing and independently validating the three route adapters. The existing manifest remains `proposed_not_executed`, with live execution disabled and provider/model fields unset. The following design must be frozen before collecting confirmatory data.

Use a finite task bank stratified by route, difficulty, satisfiable versus obstructed outcome, and vocabulary relabeling. Reserve task families and a second model for held-out evaluation. Discovery examples, admission examples, pilot examples, and confirmatory examples must be disjoint at the underlying instance level. Surface renaming alone is insufficient to make a new task.

The six planned arms are plain proposal (P), typed protocol (T), structured coverage guidance (C), generic reflection (R), admitted detector policy with feedback (D), and solution-bearing assistance (A). Every arm uses the same output schema, route checker, stopping budget, and task inputs. Arm A is an assisted control and is excluded from claims about unassisted solution finding. The principal contrast is D minus R: the combined detector policy and its feedback, relative to generic reflection.

Randomize arm order within fixture/model blocks and record the assignment. Model revisions, provider route, temperature, seed support, prompt templates, detector bank, checker bytes, retry policy, and budgets must be pinned. Unsupported seeds are reported as unsupported; repeated

requests then remain stochastic trials. The pilot estimates resource requirements and outcome variation. It does not determine success thresholds after seeing the confirmatory result.

The primary effectiveness outcome is a run-level binary variable: an independently verified route witness obtained within the common budget. Unknown, refused, timed-out, and exhausted runs are retained in the denominator and reported separately. Verification errors are not silently converted into incorrect model answers. A checker defect discovered during collection stops the affected analysis and requires a new source-bound run.

The confirmatory analysis should estimate the D–R difference using fixture blocks as the dependence unit. A prespecified paired or randomization analysis must respect the actual allocation. Report route strata, model strata, the difference, and its uncertainty. The effectiveness gate is a strictly positive lower endpoint of a 95% interval for the principal held-out comparison. Exploratory comparisons are labeled exploratory, with their multiplicity reported.

7. Safety, transfer, and stopping criteria

False suppression is evaluated per independently valid, detector-eligible proposal. In D, log proposals blocked by the enforcement policy and evaluate them with the same route oracle under a safe shadow path. In R, run the frozen detector in shadow mode on observed proposals without changing feedback. Record eligibility and whether the independent oracle accepts; do not infer validity from a model's declaration.

For every route proposed for use, require an upper 95% confidence bound below 1% on false suppression and no false terminal certificate. A route without enough valid eligible proposals is unassessed. Effectiveness on one route cannot substitute for safety on another. Adversarial near misses supplement observed proposal streams but must have separate denominators.

The sample-size issue is concrete. Under independent Bernoulli trials, observing zero failures in n opportunities gives the one-sided exact upper bound `1 - 0.05^(1/n)`, obtained by solving `(1-p)^n = 0.05`. Eight zero-failure observations leave a bound near 31%; 299 independent opportunities are needed to bring it below 1%. Shared fixtures and correlated proposals invalidate a naive independence assumption, so these numbers are an illustration of the planning burden, not this study's achieved safety result.

Held-out transfer evaluates the frozen detector bank without editing its rules. Report failures caused by new task families and model behavior as well as successes. If a new detector is added, restart the relevant held-out assessment. Stop a run on its declared call, token, time, or monetary budget. Stop the study on a false terminal, compromised source binding, or loss of required custody records. Budget exhaustion remains an observed outcome.

8. Evidence custody and reproducibility

The historical pilot file in this package, `demos/demo_3/results/live_openrouter_trial_receipt.json`, contains descriptive arm counts and provider fields for one model and two arms. Its raw request/source bindings do not establish the frozen randomized experiment above, and its trial records report terminal certificates from an earlier runner that the audited host cannot issue. Consequently this paper reports no live effect estimate, confidence interval, cost advantage, or transfer result from it.

A confirmatory receipt must bind the task, detector bank, checker, assignment, model route, pa-

rameters, raw authorized proposal bytes, decisions, state transitions, and measured resource use. Retention requires the active data policy; refusing a proposal does not authorize keeping secrets or personal data. A hash supports checking recorded bytes against an independently held anchor. It does not authenticate a provider or prove the semantic verdict.

8.1 Package checks

From the package root, with Python 3.12, run each test file separately:

```
py -3.12 -m pytest code/tests/test_demo_3_host.py -q
py -3.12 -m pytest code/tests/test_danger_gate.py -q
py -3.12 -m pytest code/tests/test_two_signals_one_detector.py -q
py -3.12 -m ruff check code
cargo test -j 1 --manifest-path code/rust/Cargo.toml
```

The Rust-versus-Python comparison in code/tests/test_danger_gate.py runs only when the binary has been built (`cargo build --release -j 1 --manifest-path code/rust/Cargo.toml`) or its path is given in `PFP003_HOST_DETECTOR_BIN`; otherwise it is skipped with that reason. These checks exercise the package re-implementation of Section 4.1, not the canonical host.

8.2 Repository checks (requires the full repository checkout)

The following commands require the full repository checkout at the baseline commit; they are not runnable from this deposit. Collect the focused tests before execution:

```
python -m pytest omega_engine_symbol_game/tests/mssn_sn_demos/test_demo_3.py --collect-
    only -q
python -m pytest omega_engine_symbol_game/tests/mssn_sn_demos/test_demo_3.py -q
python -m pytest omega_engine_symbol_game/tests/test_immune_defect_regressions.py -q
```

The Demo 3 tests include the refusal, rollback, fuel, and matrix receipt checks described in Sections 4 and 5. The two-signal conjunction example is the test test_invariant_two_signal_danger_gat e_conjunction_logic in omega_engine_symbol_game/tests/mssn_sn_demos/test_demo_imm une_soviet.py. The catalog integrity suite is omega_engine_symbol_game/tests/test_mssn_cat alog.py. Its offline fixture blocks sockets and distinguishes reproducible fixtures from live receipts whose verification is integrity-only. Use a fresh writable temporary directory; record the interpreter, collection status, command exit, and source digests. These commands are a reproducibility plan; current results must be read from the associated execution receipt.

9. Threat model and limitations

An untrusted proposer may supply a verification flag, reuse a stale witness, name nonexistent coverage, or exploit a missing adapter. An erroneous detector may reject valid novelty. A defective host may accept a malformed transition. The protocol addresses these threats through typed inputs, negative examples, independent route outcomes, and recorded refusals, each within its tested scope.

It does not assume that all host code is correct solely because it is outside the model. The canonical route seam is currently missing; signal evidence is supplied to the filter; the self panel has limited diversity; the four phrase detectors are unadmitted and cannot see negation or context; and illustrative package tests are not integration evidence. Formal models of fuel or authority apply only to their stated types and do not prove correspondence to the Python host.

The resulting contribution is a concrete admission and evaluation method with inspectable implementation boundaries. Whether admitted detectors improve solution finding safely remains an empirical question with explicit conditions for an answer. This paper assigns no authorization and changes no jurisdiction. It carries no operational authority or institutional standing.

Verification for this revision

The package verifier (`verify_deposit.py`) checks the manifest and the two-way file census, and `tools/rebuild_release.py --check` checks the release archive against the manifests. The package tests and lint are executed by the dated review receipt recorded with this revision; that receipt lists each command, tool version, exit status, and tested scope. Historical receipts in this package keep their original dates and scope and are not current execution evidence.

Appendix A. Two signals, one detector

Section 3 describes two safeguards. A pattern match (Signal 1) should refuse a proposal only when a structural defect (Signal 2) corroborates it, and a candidate detector should be screened against a self panel of strings treated as valid. Each safeguard rests on an assumption that is easy to lose in an implementation. This appendix states three small lemmas that make the assumptions exact, checks each one exhaustively on finite domains, audits the repository against them, and describes one component that realizes the admission record of Section 3.

The checks are in `code/detector_lemmas.py` and code/tests/test_two_signals_one_detector.py. They use exact integers and fractions, refuse floats and Booleans, and import nothing from the repository. Unless stated otherwise, repository paths in this appendix refer to commit 30609de2c3928238715375a9bac608256fa381d8, the revision audited on 2026-09-28, not to the code baseline of the rest of the paper.

A.1 The decision table

Write m for the number of Signal 1 matches and s for the Signal 2 record, which counts as present when its defect count is positive. The table of Section 3, as implemented in the repository gate (omega_engine_epistemic_immune_system/domain/danger_gate.py), gives `TOLERATED` when neither signal is present and `DIAGNOSTIC_ONLY` when only Signal 1 is. When only Signal 2 is present it gives `DEFENSIVE_LOCKDOWN` for a high or critical defect and `DIAGNOSTIC_ONLY` otherwise. When both are present it gives `QUARANTINED` for a high or critical defect and `DEFENSIVE_LOCKDOWN` otherwise. `DIAGNOSTIC_ONLY` is the outcome the design exists for: a match alone is recorded but refuses nothing.

A.2 Lemma A: a derived second signal collapses the table

Lemma A. Suppose Signal 2 is computed from Signal 1, $s = g(m)$ for some function g. Then:

1. the decision is a function of m alone, so two proposals with the same number of matches receive the same decision whatever their structural defects;
2. `DIAGNOSTIC_ONLY` is reachable exactly when, for some $m \geq 1$, $g(m)$ has no positive defect, or $g(0)$ is a positive defect of low or medium severity;
3. if Signal 2 is present exactly when Signal 1 is, with one severity σ for every $m \geq 1$, then exactly two decisions are reachable: `TOLERATED` and `QUARANTINED` when σ is high or critical, and `TOLERATED` and `DEFENSIVE_LOCKDOWN` otherwise.

Proof. Part 1 is substitution. For part 2, the table yields `DIAGNOSTIC_ONLY` only in the cell where Signal 1 is present and Signal 2 is absent, and in the cell where Signal 2 alone is present with low or medium severity; a derived Signal 2 reaches these cells exactly under the stated conditions. For part 3, the hypothesis leaves only the cells where both signals are absent ($m = 0$) and where both are present with severity σ ($m \geq 1$), and both cells are reached. □

Under part 3 with a high severity the gate refuses exactly when Signal 1 is present. It is then a one-signal detector, and a match can no longer be recorded without refusing. The test test_deriv ed_signal2_exhaustive_over_every_function_of_the_match_count enumerates all $13^4 = 28{,}561$ functions from the match counts $\{0, 1, 2, 3\}$ to the 13 Signal 2 values (none, or one of four severities with 0, 1 or 2 defects). It confirms part 2 for every function and part 3 for the 160 functions that satisfy its hypothesis. With the two signals chosen independently all four decisions occur (test_ind ependent_signals_reach_all_four_decisions), and the package's own table of Section 4.1 collapses in the same way (test_package_gate_collapses_the_same_way_under_a_derived_signal2).

A.3 Lemma B: the Hill-type "quorum" is a sum threshold

The repository also has a gate described as a cooperative quorum (omega_engine_epistemic_i mmune_system/domain/allostery.py). It adds the strictly positive sensor signals into $\theta = \sum_i s_i$, computes the Hill ratio $H(\theta) = \theta^n/(K^n + \theta^n)$ with $H(0) = 0$, and activates when $H(\theta) \geq t$.

Lemma B. Let $n > 0$ and $K > 0$.

1. For $0 < t < 1$ the gate activates exactly when $\theta \geq \theta^* = K\,(t/(1-t))^{1/n}$.
2. At $t = 1/2$, $\theta^* = K$ for every n, so the Hill coefficient does not affect the decision. For other t, n only moves the threshold: the gate with parameters (n, K, t) decides exactly as the gate with parameters $(1, \theta^*, 1/2)$.
3. The gate implies no number of agreeing sensors. One signal of size θ^* activates it. Only if every signal is at most u does activation require at least $\lceil \theta^*/u \rceil$ positive signals.
4. At $t = 0$ it activates with no signal at all, and at $t = 1$ it never activates.

Proof. For $\theta > 0$, $H(\theta) \geq t$ holds exactly when $\theta^n(1-t) \geq tK^n$, that is when $\theta^n \geq K^n t/(1-t)$; both sides are positive and $x \mapsto x^{1/n}$ is increasing, which gives part 1. At $\theta = 0$, $H = 0 < t$. Part 2 follows because $t/(1-t) = 1$ at $t = 1/2$, and because part 1 describes both gates by the same threshold. For part 3, m signals each at most u sum to at most mu. For part 4, $H(0) = 0 \geq 0$, and $H(\theta) < 1$ for every finite θ. □

The tests check the lemma with exact fractions for integer n from 1 to 4, four values of K and five of t, over all 400 signal vectors of length at most 3 drawn from seven values, including 0 and a negative value: the ratio computed as a fraction agrees with the cleared-denominator threshold, the decision depends only on θ and increases with it, a single sensor activates every threshold, and the bounded-sensor count holds. At $t = 1/2$ the decisions agree for $n = 1, \ldots, 6$. The repository's defaults are $n = 5/2$, $K = 2$ and $t = 1/2$; by part 2 the gate then activates when the positive signals sum to at least 2, whatever n is.

A.4 Lemma C: negative selection screens what it screens

A detector rule has a *screened* predicate P, which the screen tests against the self panel S, and an *executed* predicate E, which the matcher computes on the content it inspects. The screen admits the rule when no $x \in S$ satisfies P. Screening is *sound* for S if an admitted rule never fires on a member of S.

Lemma C. Over a universe U of inputs:

1. screening is sound for every finite panel $S \subseteq U$ exactly when $E(x)$ implies $P(x)$ for every $x \in U$;
2. screening never rejects a rule that would not fire on the panel exactly when $P(x)$ implies $E(x)$ for every $x \in U$;
3. both hold for every panel exactly when $E = P$ on U.

Proof. For part 1, if E implies P and the rule is admitted, no member of S satisfies P, so none satisfies E. Conversely, if some x has $E(x)$ but not $P(x)$, the panel $S = \{x\}$ admits the rule, which then fires on x. Part 2 is the same argument with E and P exchanged, and part 3 combines the two. □

The test test_screening_is_sound_for_every_panel_iff_executed_implies_screened checks parts 1 and 2 for all 64×64 pairs of predicates on a six-element universe and all 63 nonempty panels. The test test_pattern_ignoring_matcher_is_admitted_and_fires_on_a_self_fixture gives the counterexample of part 1 in the form found below: the screen tests a failure-class name, the matcher tests the structure of the record, and the rule is admitted and fires on a member of the panel.

A.5 The repository audited against the lemmas

The findings below refer to omega_engine_epistemic_immune_system/ at the audited commit.

Lemma A. The packaged demonstration use_cases/demos/demo_4e_cogsci_autopoiesis/run_packaged_immune.py computes Signal 2 for each of its 24 control cases from the match count: line 178 passes `{"defect_count": len(matches)}` to a sensor that reports a high-severity defect whenever the count is positive (infrastructure/mssn_ports.py, lines 27 to 39). Signal 1 is then recomputed by the same matcher over the rules that the screen admits. When the screen admits every rule, part 3 of Lemma A applies: the only reachable decisions are `TOLERATED` and `QUARANTINED`, and every match refuses. The demonstration's other path, for model replies, takes Signal 2 from a separate defect check and is not affected. Correction: in commit `fdc0035a8` on branch claude/mssn-study-corrections, Signal 2 comes from host checks that the matcher never sees (`domain/host_checks.py` and `domain/danger_signals.py`), assert_signals_independent refuses a derived Signal 2 with the code `TWO_SIGNAL_COLLAPSE`, and the gate follows the two-signal rule, under which Signal 2 alone is diagnostic only (tests/unit/test_two_signal_independence.py).

Lemma B. `domain/allostery.py`, lines 74 to 79, computes the gate of Section A.3 in floating point. At its defaults it activates when the positive signals sum to at least 2, and a single sensor suffices. The comment at line 68 says that $K = 2$ "requires 2 equivalent unit signals"; that holds only if each signal is at most 1, which the code does not require. With $t = 0$ it activates on an empty list. Correction: in commit `fdc0035a8` the gate is exact, with an integer Hill coefficient, a threshold strictly between 0 and 1, a cap of 1 on each signal, so that at the defaults at least two sensors are needed, and typed refusals (tests/unit/test_allostery_exact.py).

Lemma C. The screen in `domain/thymus.py`, lines 77 to 99, compiles each rule's regular expression and searches the self panel with it. Three of the four matchers behind the same port (`PharmacyMatcher`, `FarmMatcher` and `ProtocolMatcher` in infrastructure/mssn_ports.py, lines 80 to 166) never execute that expression: they use a rule only to select a target class and report the findings of a separate detector. For the pharmacy rules the screened expression is the escaped name of the failure class (`run_packaged_immune.py`, lines 118 to 122). The executed and screened predicates

differ, so by Lemma C the screen gives these rules no soundness: a self fixture that the detector flags but that does not contain the class name passes the screen. Correction: in commit `fdc0035a8` each rule binds the identifier of the matcher that executes it, the screen runs that matcher on the self panel, and regular expressions are limited to a subset that cannot backtrack without bound (`domain/safe_pattern.py`; tests/unit/test_thymus_binds_executed_matcher.py).

The package re-implementation of Section 4.1 is not affected by the last two findings; its table counts any supplied Signal 2 record as present, and Lemma A applies to it whenever that record is computed from the matches.

A.6 A realized admission record: anti-skill V2

Section 3 asks that a detector be admitted only with a reproduced defect, valid near misses, a frozen predicate and bound digests. One component of the repository states that record as types: anti-skill V2, in omega_engine_workflows_agents_skills/domain/anti_skill/anti_skill_v2.py and application/anti_skill/anti_skill_v2.py of the same package.

- *A closed predicate language.* A detector is an expression of at most 16 atoms, combined by `ALL` or `ANY`. Each atom applies one of four operators (`EQUAL_FIELDS`, `NOT_SUBSET_FIELDS`, `VALUE_IN` , `IS_EMPTY`) to eight declared fields of a subject record. There is no free-form code, regular expression or model verdict, so the predicate that is evaluated is the predicate that is recorded: the condition $E = P$ of Lemma C holds by construction.
- *Positive and negative controls in at least two contexts.* Evaluation refuses a specification whose fixture bank lacks an expected hit, lacks a negative control, or draws its cases from fewer than two contexts (application module, lines 140 to 148). A negative control cannot expect a hit, and copied subjects are refused.
- *Derived receipts.* An evaluation receipt is refused unless its outcome equals the outcome computed from its case results, and every record's identifier is recomputed from its canonical content.
- *A human decision.* Admission requires a closed `APPROVE` or `REJECT` decision bound to the specification digest and to the evaluation receipt, and a verified registry requires one approving decision per specification, matched one to one with a passing receipt.

At the audited commit three specifications, their receipts and three approving decisions exist in a registry scoped to one engine profile (claim ledger row P3-26 gives the paths). The Python admission verifier and the loader that would activate that registry are retired and return a typed refusal (omega_engine_workflows_agents_skills/application/anti_skill_admission/verifier.py, lines 25 to 45), no Rust code reads the registry, and the registry that the prompt reviewer loads is empty. No detector is therefore admitted and active through this record. It shows that the admission record of Section 3 can be written as types whose outcomes are derived rather than declared; whether admitted detectors help remains the experiment of Sections 6 and 7. The corrections of 2026-09-29 leave anti-skill V2 unchanged; moving it into the immune-system package, where the corrected gate and screen now live, is recorded as open work.

References

- Chandrasekharan, S., and Wiese, U.-J. (1999). Meron-cluster solution of fermion sign problems. *Physical Review Letters* 83, 3116–3119. doi:10.1103/PhysRevLett.83.3116; arXiv:cond-mat/9902128.

- Li, Z.-X., Jiang, Y.-F., and Yao, H. (2016). Majorana-time-reversal symmetries: a fundamental principle for sign-problem-free quantum Monte Carlo simulations. *Physical Review Letters* 117, 267002. doi:10.1103/PhysRevLett.117.267002; arXiv:1601.05780.
- Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Majumder, B. P., Hermann, K., Welleck, S., Yazdanbakhsh, A., and Clark, P. (2023). Self-Refine: Iterative Refinement with Self-Feedback. arXiv preprint 2303.17651, version 2.
- Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., and Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv preprint 2303.11366, version 4.
- Troyer, M., and Wiese, U.-J. (2005). Computational complexity and fundamental limitations to fermionic quantum Monte Carlo simulations. *Physical Review Letters* 94, 170201. doi:10.1103/PhysRevLett.94.170201; arXiv:cond-mat/0408370.
- Wei, Z. C., Wu, C., Li, Y., Zhang, S., and Xiang, T. (2016). Majorana positivity and the fermion sign problem of quantum Monte Carlo simulations. *Physical Review Letters* 116, 250601. doi:10.1103/PhysRevLett.116.250601; arXiv:1601.01994.

Implementation links throughout the paper identify files at the stated source baseline. The proposed experiment manifest is a separate design artifact.